

Part 1: The Knapsack Problem (BFS & DFS)

The Knapsack Problem (BKP) is a common problem within combinatorial optimization. To solve complex search problems, it introduces weight and benefits to maximize utility within some given constraints.

Objective

Your task is to solve the **0-1 Knapsack Problem**. You are given a set of items, each with a specific weight and benefit. You must determine which items to include in a knapsack so that the total weight does not exceed the capacity, while maximizing the total benefit.

In the 0-1 variant, you cannot break items apart. You must either take the whole item (1) or leave it (0).

Data and Files

- **File:** assignment 1 knapsack.txt (Available on Canvas).
- **Format:** The file contains a list of items. Each row represents an item with an **Item ID**, **Benefit (b)**, and **Weight (w)**.

Below is a representation of items and details:

Item ID	Benefit (b)	Weight (w)
1	20	15
2	40	32
...	...	...

The Task

Implement **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** to find the optimal combination of items.

- Conceptualize the search space as a tree where each level represents a decision for an item (Take it or Leave it).
 - **BFS:** Explore the tree layer by layer using a Queue.
 - **DFS:** Explore the tree branch by branch using a Stack.
-

Part 2: Pathfinding in Spain (Greedy & A*)

Objective

Instead of the Romania map used in lectures, we will use a map of **Spain**. Your goal is to find a path from **MALAGA**(Start Node) to **VALLADOLID** (Goal Node).

Data and Files

- **File:** Assignment 1 Spain map.txt (Available on Canvas).
- **Content:**
 - **Connections:** Costs (distances) between connected cities.
 - **Heuristic Data:** Straight-Line Distance (SLD) from each city to the goal (Valladolid).

Example data:

City A	City B	Cost
City A	City C	Cost
...		
City A	Straight_Line_Distance_to_goal	
City B	Staright_Line_Distance_to_goal	
...		

The Task

Implement **Greedy Best-First Search** and *A Search** to find the path.

- **Greedy Best-First Search:** Should use only the heuristic ($h(n)$) to select the next node. It generally finds *a* solution quickly but is not guaranteed to find the optimal one.
- *A Search*:* Should use both the path cost so far ($g(n)$) and the heuristic ($h(n)$). It must guarantee the optimal (shortest) path.